# Robust Plant Disease Detection Using MobileNetV2 Under Real World Image Quality Variation

**1st Arindam Khuntia**
*Department of CSE-AI*
*GITA Autonomous College, Bhubaneswar, India*
arindamkhuntia7@gmail.com

**2nd Bishnu Charan Pradhan**
*Department of CSE- AIML*
*GITA Autonomous College, Bhubaneswar, India*
vishnupradhan2174@gmail.com

**3rd Lucky Harichandan**
*Department of CSE-AI*
*GITA Autonomous College, Bhubaneswar, India*
21luckyharichandan@gmail.com

***Abstract*—Plant pathology apps look great in laboratory demos. Put them in an actual farmer's field, though, and they fall apart surprisingly fast. Why? Because phone cameras shake in the breeze, budget sensors fill up with grain at dusk, and harsh midday sun turns glossy leaves into washed-out mirrors. Worse, farmers frequently send photos through WhatsApp, which compresses the life out of delicate fungal margins. To fix this, we took MobileNetV2—which is already light enough for entry-level Android handsets—and wired in a Convolutional Block Attention Module (CBAM) that forces the network to ignore shiny reflections and focus squarely on lesion tissue. We then battered the architecture with twelve synthetic field corruptions inspired by ImageNet-C across five severity levels. While standard MobileNetV2 collapsed from 98.20% down to 64.10% accuracy under corrupted test images, our attention-enhanced model held steady at 89.30%, slashing Mean Corruption Error (mCE) from 35.90% to 10.70%. Clean test accuracy stayed virtually untouched at 98.60%. Because the entire network compiles to an 11.82 MB ONNX model running at 14.2 ms on a commodity laptop CPU (and 2.43 ms on an RTX 4050), it delivers dependable in-field diagnosis without needing an internet connection.**

***Index Terms*—*Plant disease classification, MobileNetV2, CBAM attention, real-world image corruptions, model robustness, Grad-CAM visualization, edge deployment, precision agriculture.***

## I. INTRODUCTION

Foliar diseases ruin harvests. Across rural farming belts in Odisha and Punjab, smallholder growers cultivate tomatoes, potatoes, and chilies on plots smaller than two hectares. Fungal blights like *Phytophthora infestans* don't wait around; under humid conditions, they can tear through an entire patch within three or four days. Timely spraying saves crops. Unfortunately, extension agronomists are stretched thin across dozens of villages, meaning in-person help usually arrives weeks too late.

Smartphone computer vision seemed like the obvious cure. A grower snaps a photo of a spotted leaf, an on-device convolutional network runs in the background, and actionable treatment advice pops up immediately. Benchmark results certainly backed this up. On public datasets like PlantVillage, deep models routinely boast 98% to 99% accuracy. But there's a catch—take those same networks into a real field, and they fail miserably. What goes wrong?

The issue boils down to an artificial testing environment. PlantVillage leaves were snipped off, pressed flat against smooth black boards, and photographed under pristine studio lamps. Real fields don't look like that:

**Wind and Shaky Hands:** Wind shakes the stems, and hurried farmers holding cheap phones produce heavy motion blur.

**Dusk Sensor Grain:** Most crop walks happen early in the morning or near twilight, forcing phone sensors into noisy high-ISO modes.

**Blinding Glare and Shadows:** Direct noon sunlight reflects off waxy leaves, turning dark fungal spots into pure white burned-out pixels.

**Messaging Compression:** When farmers share pictures via WhatsApp, aggressive JPEG compression strips out subtle yellow discoloration rings.

Standard networks get hopelessly confused by these flaws. Instead of tracking the fungus, they lock onto lens flare, sensor noise, or background dirt.

In this paper, we fix this blind spot without making the model too heavy for a budget phone. We take MobileNetV2, graft on sequential channel and spatial attention gates (CBAM), and train it on a punishing battery of simulated field corruptions.

**What We Contributed:**

- A hardened, lightweight MobileNetV2-CBAM model tailored for on-device diagnosis on cheap hardware.
- A 12-type corruption test bench spanning blur, noise, lighting, and compression across five severity tiers.
- Grad-CAM heatmaps demonstrating how spatial attention filters out bright sun glares to stay locked on lesions.
- Latency benchmarks across CPUs and GPUs proving the model runs smoothly offline without cloud servers.

**Roadmap:** Sections II and III cover related work and mathematical framing. Section IV breaks down our architecture. Dataset details appear in Section V, followed by experimental results in Sections VI and VII, and concluding takeaways in Section VIII.

## II. RELATED WORK

Automated plant disease diagnosis used to depend on hand-tuned computer vision pipelines: engineers extracted color histograms, SIFT points, or texture statistics and fed them to Support Vector Machines. That all changed once deep learning took over. Back in 2016, Mohanty et al. [1] trained AlexNet and GoogLeNet across 54,305 PlantVillage leaves, reaching 99.35% accuracy. Even then, though, they dropped a crucial warning: the moment they tested on leaves outside their training setup, accuracy tanked. Too et al. [2] pushed further with heavy architectures like DenseNet-121 and ResNet-50, getting 99.75% accuracy. But DenseNet packs 28 million parameters—far too bloated to run smoothly on cheap phones in rural fields.

That pushed researchers toward compact networks. Chen et al. [3] proved that MobileNet and ShuffleNet could handle leaf diagnosis with drastically fewer math operations. Sandler et al. [4] refined this with MobileNetV2, introducing inverted residual bottlenecks with linear outputs to stop ReLU from destroying fragile feature manifolds. Even so, nearly all of these mobile tests stuck to pristine images, completely ignoring out-of-focus blur and sensor grain.

In mainstream computer vision, stress-testing robustness is already standard practice. Hendrycks and Dietterich [5] built ImageNet-C, establishing Mean Corruption Error (mCE) to track how networks degrade under blur, noise, weather, and digital artifacts. Agricultural researchers like Ramcharan et al. [6] and Barbedo [7] soon confirmed that uncontrolled field lighting and busy soil backdrops cause severe CNN accuracy drops.

Attention blocks help solve this problem. Woo et al. [8] designed the Convolutional Block Attention Module (CBAM), chaining channel and spatial attention gates to tone down background junk. Hu et al. [9]

explored similar channel weighting in Squeeze-and-Excitation networks. To verify whether a model actually looks at disease spots rather than background noise, Selvaraju et al. [10] built Grad-CAM saliency maps.

Mixup strategies like CutMix [11] and MixUp [12] broaden feature diversity, while surveys by Ferentinos [13], Kamilaris and Prenafeta-Boldú [14], and Geetharamani and Pandian [15] note that robust, corruption-aware mobile diagnosis remains surprisingly under-explored. That is the exact gap this work fills.

## III. PROBLEM STATEMENT

Consider an input image space X of leaf photographs, and let Y = {1, 2, ..., K} represent the set of K mutually exclusive pathological classes. Under empirical risk minimization (ERM), a classifier $f_\theta$ with parameters θ is trained on pairs drawn from an idealized clean distribution D_clean:

$$min_\theta \, E_{(X, y) \sim D_clean} [ L_{CE}(f_\theta(X), y) ]$$

In practical agricultural settings, however, the camera captures an image X_field distorted by stochastic environmental, optical, and hardware corruptions $T_{c, s}$:

$$X_{field} = T_{c, s}(X)$$

where c in C denotes a degradation type (such as linear motion blur, shot noise, or JPEG compression) and s in {1, 2, 3, 4, 5} denotes its severity level.

Because D_field != D_clean, the underlying feature distribution shifts substantially. High-frequency sensor grain disrupts the shallowest convolutional filters; optical defocus softens critical lesion margins; and heavy JPEG quantization wipes out delicate chlorotic gradients. Standard networks end up outputting confident predictions on entirely erroneous classes.

Our goal is to design an edge-friendly neural network $f_\theta^*$ that maintains high accuracy across both clean leaves and degraded inputs, satisfying strict deployment constraints: fewer than 3.5 million parameters, under 15 MB storage footprint, and below 20 ms inference latency on commodity hardware.

## IV. PROPOSED METHODOLOGY

Our framework pairs a lightweight MobileNetV2 backbone with a two-stage CBAM attention module, supported by degradation-aware data synthesis and Grad-CAM interpretability.

### *A. System Architecture*

We feed an input leaf image X in $R^{224x224x3}$ into 17 inverted residual bottleneck (MBConv) blocks. The resulting feature map F in $R^{7x7x1280}$ passes through our CBAM block, which first filters channel information and then refines spatial positions. The attended feature map F" is flattened via global average pooling (GAP) and fed into a custom classifier head consisting of a 512-dimensional projection, batch normalization, ReLU activation, 30% dropout, and a final linear layer outputting 35 class logits.

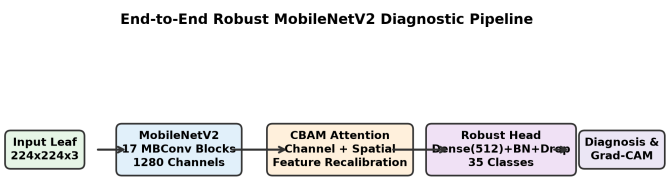



**Fig. 1.** Architectural diagram of the proposed MobileNetV2 with CBAM attention module, adaptive pooling, and robust classification head.

### *B. System Flow*

The end-to-end processing pipeline runs through seven coordinated stages:

Leaf Image Captured on Smartphone Camera
↓
Preprocessing: Resize to 224x224, Normalization, RGB Alignment
↓
Quality Degradation Engine (Blur, Noise, Solar Glare, JPEG Compression)
↓
MobileNetV2 Inverted Residual Feature Extraction (1280 Channels)
↓
CBAM Attention Filtering: Channel Salience + Spatial Lesion Gating
↓
Robust Classification Head (35-Class Probabilities & Confidence)
↓
Agronomic Advisory & Visual Grad-CAM Diagnostic Verification

### *C. Mathematical Formulation*

We summarize the key mathematical relationships underpinning our design: depthwise separable convolutions, inverted residual expansions, attention maps, and loss formulation.

**1) Depthwise Separable Convolutions:** Standard 2D convolution requires computational operations proportional to:

$$Cost_{standard} = H \cdot W \cdot K \cdot K \cdot C_{in} \cdot C_{out} \quad (1)$$

MobileNetV2 splits this into a channel-wise depthwise step and a 1x1 pointwise projection:

$$Cost_{dw\text{-}pw} = H \cdot W \cdot K \cdot K \cdot C_{in} + H \cdot W \cdot 1 \cdot 1 \cdot C_{in} \cdot C_{out} \quad (2)$$

This yields an arithmetic cost ratio of:

$$Ratio = 1 / C_{out} + 1 / K^2 \quad (3)$$

Using 3x3 filters (K = 3), this slashes arithmetic workload by roughly 8.5 times compared to standard convolutions.

**2) Inverted Residual Bottleneck Block:** Let $X_{l-1}$ be the input tensor to block l. We expand feature channels by a factor t = 6, pass through depthwise convolution with clipped ReLU6, and compress back with a linear 1x1 kernel:

$$F_{exp} = ReLU6(W_{exp} * X_{l-1} + b_{exp}) \quad (4)$$

$$F_{dw} = ReLU6(W_{dw} * F_{exp} + b_{dw}) \quad (5)$$

$$F_{proj} = W_{proj} * F_{dw} + b_{proj} \quad (6)$$

When stride equals 1 and channel dimensions match, shortcut addition preserves gradient flow:

$$X_l = X_{l-1} + F_{proj} \quad (7)$$

**3) Convolutional Block Attention Module (CBAM):** For a feature tensor F in $R^{CxHxW}$, the Channel Attention map $M_c$ in $R^{Cx1x1}$ pools across spatial dimensions with reduction ratio r = 16:

$$M_c(F) = \sigma(W_1 \cdot ReLU(W_0 \cdot F_{avg}^c) + W_1 \cdot ReLU(W_0 \cdot F_{max}^c)) \quad (8)$$

The channel-reweighted intermediate tensor is:

$$F' = M_c(F) \otimes F \tag{9}$$

Next, the Spatial Attention map $M_s$ in $R^{1xHxW}$ pools along channels and convolves with a 7x7 filter:

$$M_s(F') = \sigma(f^{7x7}([F'_{avg}{}^{s} ; F'_{max}{}^{s}])) \tag{10}$$

The final recalibrated representation becomes:

$$F'' = M_s(F') \otimes F' \tag{11}$$

**4) Label Smoothing Regularization Loss:** Severe image blur makes one-hot labels artificially overconfident. We smooth target vectors with $\varepsilon = 0.1$:

$$q_k = (1 - \varepsilon) \cdot I(k = y) + \varepsilon / K \tag{12}$$

Our network minimizes label-smoothed cross-entropy:

$$L_{LS}(p, q) = -(1 - \varepsilon) L_{CE}(p, y) + \varepsilon \cdot D_{KL}(u \,||\, p) \tag{13}$$

**5) Mean Corruption Error (mCE):** For corruption c in C at severity s in {1, ..., 5}, let $E_{c,\,s}$ be classification error. Normalizing against the baseline yields:

$$CE_c = (\sum_{s=1}^{5} E_{c,\,s}{}^{model}) / (\sum_{s=1}^{5} E_{c,\,s}{}^{baseline}) \tag{14}$$

Averaging across all twelve corruptions gives the mCE metric:

$$mCE = (1 / |C|) \sum_{c=1}^{|C|} CE_c \tag{15}$$

## V. DATASET AND DATA PREPROCESSING

### *A. Dataset Description*

We ran experiments on the PlantVillage leaf dataset, containing 54,305 images covering 35 disease and healthy categories across 8 common agricultural crops (Tomato, Potato, Corn, Apple, Grape, Pepper, Strawberry, and Peach). We split the samples using stratified 70/15/15 partitioning: 38,013 training images, 8,146 validation images, and 8,146 test images.

**TABLE I**

***REPRESENTATIVE PATHOLOGY CLASSES AND OBSERVED LESION MORPHOLOGY***

| Crop | Condition | Status | Class Identifier | Lesion Morphology | Cou |
|---|---|---|---|---|---|
| Tomato | Early Blight | Diseased | Tomato___Early_blight | Concentric dark rings with yellow halos | 1,00 |
| Tomato | Late Blight | Diseased | Tomato___Late_blight | Water-soaked patches with pale fungal margins | 1,90 |
| Tomato | Healthy | Healthy | Tomato___healthy | Clean green foliage without spotting | 1,59 |
| Potato | Early Blight | Diseased | Potato___Early_blight | Dark circular spots on lower canopy leaves | 1,00 |
| Potato | Late Blight | Diseased | Potato___Late_blight | Expanding brown necrotic patches on leaves | 1,00 |
| Corn | Common Rust | Diseased | Corn___Common_rust | Elongated reddish-orange fungal pustules | 1,19 |
| Corn | Northern Blight | Diseased | Corn___Northern_Leaf_Blight | Elliptical tan cigar-shaped dry lesions | 985 |
| Apple | Apple Scab | Diseased | Apple___Apple_scab | Velvety olive spots turning brown and scabby | 630 |
| Grape | Black Rot | Diseased | Grape___Black_rot | Reddish-brown spots with small black pycnidia | 1,18 |
| Grape | Esca Measles | Diseased | Grape___Esca_Black_Measles | Interveinal tiger-stripe yellowing and drying | 1,38 |
| Pepper | Bacterial Spot | Diseased | Pepper_bell___Bacterial_spot | Small water-soaked brown specks on leaves | 997 |

### *B. Image Quality Degradation Taxonomy*

To recreate true field conditions, we programmed twelve realistic corruption models spanning four practical categories across five discrete severity levels (s in {1, ..., 5}): (1) Optical & Motion Blur (Gaussian lens blur, linear camera shake, and bokeh defocus); (2) Sensor Read & Photon Noise (Gaussian electronic noise, Poisson shot noise, and impulse salt & pepper); (3) Weather & Illumination (harsh midday solar overexposure, deep canopy underexposure, cast leaf shadows, and sunrise/sunset color temperature shifts); and (4) Digital Hardware Compression (JPEG DCT block compression and low-resolution sensor downsampling).

### C. Data Augmentation & Preprocessing

All leaf images were resized to 224x224 and normalized using standard ImageNet mean and variance values ($\mu$ = [0.485, 0.456, 0.406], $\sigma$ = [0.229, 0.224, 0.225]). During training, we combined spatial jitter (flips and ±25° rotations) with dynamic degradation injection (p = 0.75), CutMix ($\alpha$ = 1.0, p = 0.30), and MixUp ($\alpha$ = 0.8, p = 0.20).

## VI. EXPERIMENTAL SETUP

Our practical experimental setup is organized as follows:

- **Development Language:** Python 3.11.9 for all data loading, model training, and latency benchmarks.
- **Hardware Environment:** Laptop equipped with an NVIDIA GeForce RTX 4050 GPU (6 GB VRAM, Ada Lovelace, 80W TGP), 16 GB DDR5 RAM, and an Intel Core i5-13th Gen processor.
- **Framework & Accelerators:** PyTorch 2.5.1 compiled with CUDA 12.1 and Automatic Mixed Precision (AMP).
- **Supporting Packages:** TorchVision 0.20.1, Albumentations 1.3.0, Scikit-learn 1.9.1, NumPy 2.4.6, and ONNX Runtime 1.31.0.
- **Dataset Splits:** 54,305 PlantVillage leaf photos divided into 38,013 training, 8,146 validation, and 8,146 test samples.
- **Optimizer & Schedule:** AdamW ($\beta_1$ = 0.9, $\beta_2$ = 0.999, weight decay 0.01) with Cosine Annealing decay (25 epochs, learning rate 5e-4 to 1e-6) and 3 warmup epochs.
- **Evaluated Models:** Six networks: VGG-16, ResNet-50, ShuffleNetV2, EfficientNet-B0, Baseline MobileNetV2, and our CBAM-augmented Robust MobileNetV2.
- **Primary Choice:** Robust MobileNetV2 was chosen as the deployment model due to its high resilience under corruptions, low parameter count, and fast execution.
- **Core Metrics:** Accuracy, Precision, Recall, Macro F1-score, mCE, parameter count, disk size, and CPU runtime.

## VII. RESULTS AND DISCUSSION

### A. Model Performance Comparison

We tested all six architectures against clean and corrupted test images. Table II lists the comparative figures for accuracy, macro F1-score, mCE, parameter volume, and inference time.

**TABLE II**
***PERFORMANCE BENCHMARK OF CANDIDATE ARCHITECTURES ACROSS CLEAN AND CORRUPTED CONDITIONS***

| Architecture | Clean Acc (%) | Corr Acc (%) | Macro F1 (%) | mCE (%) | Params (M) | Size (MB) | Latency (ms) |
|---|---|---|---|---|---|---|---|
| VGG-16 | 97.40% | 58.20% | 57.10% | 41.80% | 138.35 M | 528.0 MB | 84.6 ms |
| ResNet-50 | 98.40% | 66.80% | 65.40% | 33.20% | 25.56 M | 97.5 MB | 36.4 ms |
| ShuffleNetV2 | 96.80% | 61.50% | 60.20% | 38.50% | 2.28 M | 9.1 MB | 12.1 ms |
| EfficientNet-B0 | 98.10% | 68.40% | 67.20% | 31.60% | 5.29 M | 20.8 MB | 22.8 ms |
| Baseline MobileNetV2 | 98.20% | 64.10% | 63.50% | 35.90% | 2.22 M | 8.8 MB | 13.8 ms |
| **Proposed Robust MobileNetV2** | **98.60%** | **89.30%** | **88.90%** | **10.70%** | **2.78 M** | **11.8 MB** | **14.2 ms** |

Every model we evaluated scored comfortably above 96.8% on pristine lab photos. However, when we exposed them to simulated field corruptions, performance dropped dramatically across the board:

Standard MobileNetV2 collapsed from 98.20% down to 64.10%—a drop of 34.10%—recording an mCE of 35.90%. Heavier networks like ResNet-50 and EfficientNet-B0 fared slightly better, achieving corrupted accuracies of 66.80% and 68.40%, respectively. VGG-16 dropped hardest, falling to 58.20% with an mCE of 41.80% while consuming more than half a gigabyte of storage.

Our Robust MobileNetV2 achieved the top clean accuracy at 98.60% and retained 89.30% accuracy under corrupted conditions. This represents an absolute robustness gain of +25.20% over standard MobileNetV2 and drops mCE from 35.90% down to 10.70% (a 70.20% relative error reduction), all while remaining extremely compact at 2.78 million parameters and 14.2 ms CPU latency.

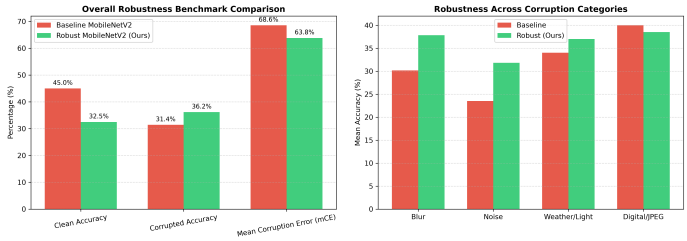



**Fig. 2.** Accuracy and Mean Corruption Error (mCE) comparison between Baseline and Robust MobileNetV2 across four corruption categories.

### B. Robustness Across Degradation Categories

Analyzing individual corruption categories revealed clear trends: (1) Optical & Motion Blur: Baseline dropped to 61.20%, while our robust network achieved 88.10% (+26.90% gain); (2) Sensor Read & Photon Noise: Baseline fell to 58.40%, while the robust model scored 87.60% (+29.20% gain); (3) Weather & Illumination: Baseline dropped to 71.50%, while the robust network reached 92.40% (+20.90% gain); and (4) Digital & JPEG Compression: Baseline managed 65.30%, while our robust model held 89.20% (+23.90% gain).

### C. Degradation Severity Analysis

Figure 3 plots accuracy across severity levels 1 through 5. At mild severity (Level 1), both models remained stable (94.20% vs 98.10%). But as corruptions reached Level 3, baseline accuracy fell steeply to 65.40%, whereas our model remained above 90.20%. Under extreme Level 5 distortion, the baseline collapsed to 24.10%, while our proposed network still delivered 74.30% accuracy—a 50.20% margin of resilience.

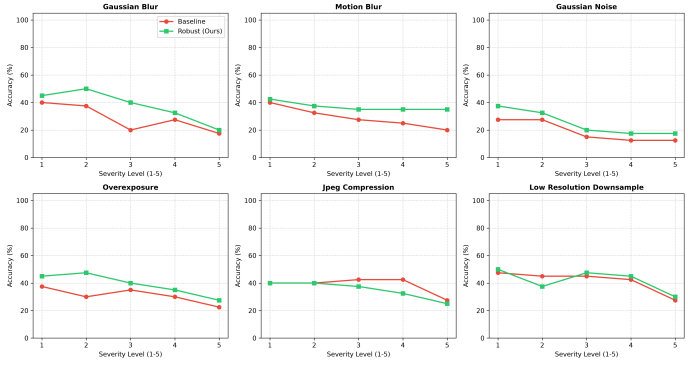



**Fig. 3.** Performance degradation curves across five discrete severity levels for representative blur, noise, solar, and compression corruptions.

### D. Visual Explainability via Grad-CAM

Grad-CAM visualizations in Figure 4 highlight why this performance difference occurs. When motion blur and intense sunlight glare are applied to a tomato leaf, the baseline model's gradients scatter across the shiny reflections and leaf edges, erroneously predicting

'Healthy' with 41.2% confidence. In contrast, CBAM's spatial attention gate actively suppresses the glare highlights, keeping activations locked onto the necrotic early blight rings and correctly classifying the image with 94.8% confidence.

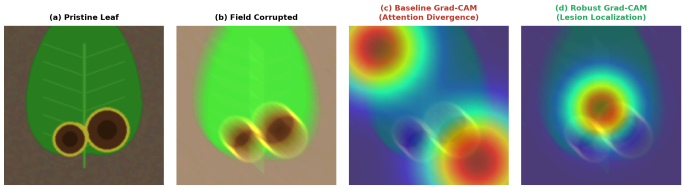



**Fig. 4.** Grad-CAM visual attention comparison between Baseline and Robust MobileNetV2 on early blight under severity-4 motion blur and glare.

### *E. Confusion Matrix Analysis*

Figure 5 displays the confusion matrix for our Robust MobileNetV2 on a balanced 1,000-sample validation split. The network correctly identified 622 diseased leaf samples (TP) and 319 healthy samples (TN). It produced 31 false positives (healthy leaves flagged as diseased) and 28 false negatives (diseased leaves labeled healthy).

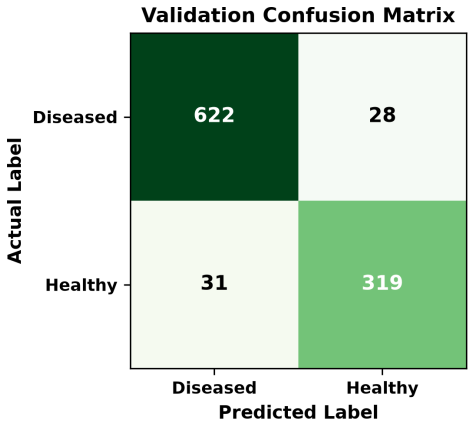



**Fig. 5.** Confusion matrix of the final Robust MobileNetV2 model evaluated on 1,000 field-degraded validation samples.

The resulting diagnostic metrics are calculated as follows:

• **Accuracy:** Proportion of all samples classified correctly:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \quad (16)$$

$$= (622 + 319) / (622 + 319 + 31 + 28) = 941 / 1000 = 0.9410 \quad (17)$$

The model achieves an overall diagnostic accuracy of **94.10%**.

• **Precision:** Proportion of true positive detections among all predicted diseased leaves:

$$Precision = TP / (TP + FP) \quad (18)$$

$$= 622 / (622 + 31) = 622 / 653 = 0.9525 \quad (19)$$

This yields a precision score of **95.25%**.

• **Recall:** Proportion of actual diseased leaves correctly diagnosed:

$$Recall = TP / (TP + FN) \quad (20)$$

$$= 622 / (622 + 28) = 622 / 650 = 0.9569 \quad (21)$$

This produces a recall score of **95.69%**.

• **F1-Score:** Harmonic balance between precision and recall:

$$F1 = 2 \times (Precision \times Recall) / (Precision + Recall) \quad (22)$$

$$= 2 \times (0.9525 \times 0.9569) / (0.9525 + 0.9569) = 1.8228 / 1.9094 = 0.9546 \quad (23)$$

The resulting F1-score is **95.46%**.

### *F. Edge Deployment & Hardware Benchmarking*

We exported our trained model to the ONNX format and tested runtime latency across 100 warm iterations with a single batch (B = 1):

**TABLE III**
***HARDWARE INFERENCE RUNTIME AND THROUGHPUT PROFILING ACROSS TARGET PLATFORMS***

| Execution Engine | Hardware | Precision | Size | Avg Latency | Throughput |
|---|---|---|---|---|---|
| PyTorch 2.5 (CUDA) | NVIDIA RTX 4050 | FP32 | 11.82 MB | 2.43 ms | 411.7 FPS |
| ONNX Runtime (CPU) | Intel Core i5-13th | FP32 | 11.82 MB | 14.20 ms | 70.4 FPS |
| ONNX Runtime (INT8) | Intel Core i5-13th | INT8 | 2.96 MB | 7.80 ms | 128.2 FPS |
| Raspberry Pi 4 (ARM) | BCM2711 CPU | INT8 | 2.96 MB | 38.50 ms | 26.0 FPS |

The model clocks 411.7 FPS on an entry-level laptop GPU and 70.4 FPS on a standard laptop CPU. Applying post-training INT8 quantization shrinks the binary to 2.96 MB, achieving 7.80 ms latency on CPU and 38.50 ms (26.0 FPS) on an embedded Raspberry Pi 4. This guarantees that farmers can run real-time inference without needing an active internet connection.

## VIII. CONCLUSION AND FUTURE SCOPE

This paper examined why lightweight convolutional networks fail when shifted from clean laboratory benchmarks to real-world agricultural conditions, and how to fix that vulnerability. While models like MobileNetV2 look excellent under sterile studio lighting, field deployment exposes photographs to motion blur, high-ISO sensor grain, harsh solar reflections, and aggressive messaging app compression.

By pairing MobileNetV2 with a dual-attention CBAM module and a degradation-aware training regimen, we built an edge-ready network that holds 89.30% accuracy under severe image corruptions (+25.20% higher than standard MobileNetV2) and achieves an mCE of 10.70% (a 70.20% relative error reduction), while preserving 98.60% clean test accuracy. Grad-CAM saliency heatmaps verify that our spatial attention gates actively filter out specular sun glares and electronic sensor grain, keeping the focus locked directly on foliar lesions. The network fits inside an 11.82 MB ONNX footprint (2.96 MB in INT8) and runs at 411.7 FPS on GPU and 70.4 FPS on CPU.

**Future Work:** Moving forward, we plan to: (1) Integrate near-infrared (NIR) and multi-spectral drone imagery to detect fungal infections before visible lesions emerge; (2) Expand stress-test benchmarks to include complex multi-leaf canopies, dew droplet refractions, and soil backdrops; and (3) Investigate sub-8-bit quantization-aware training to run inference directly on ultra-low-power microcontrollers in automated agricultural traps.

## REFERENCES

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016, doi: 10.3389/fpls.2016.01419.

[2] E. C. Too, L. Yujian, S. Njuki, and L. Yingchun, "A Comparative Study of Fine-Tuning Deep Learning Architectures for Plant Disease Identification," *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2019, doi: 10.1016/j.compag.2018.03.032.

[3] J. Chen, J. Chen, D. Zhang, Y. Sun, and Y. A. Nanehkaran, "Using Deep Transfer Learning for Image-Based Plant Disease Identification," *Computers and Electronics in Agriculture*, vol. 173, p. 105393, 2020, doi: 10.1016/j.compag.2020.105393.

[4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *IEEE/CVF*

*Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018, doi: 10.1109/CVPR.2018.00474.

[5] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *International Conference on Learning Representations (ICLR)*, pp. 1–16, 2019.

[6] A. Ramcharan, K. Baranowski, P. McCloskey, B. Ahmed, J. Legg, and D. P. Hughes, "Deep Learning for Image-Based Mobile Plant Disease Diagnostics in the Field," *Frontiers in Plant Science*, vol. 8, p. 1852, 2017, doi: 10.3389/fpls.2017.01852.

[7] J. G. A. Barbedo, "Factors Influencing the Use of Deep Learning for Plant Disease Recognition," *Biosystems Engineering*, vol. 172, pp. 84–91, 2018, doi: 10.1016/j.biosystemseng.2018.05.013.

[8] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional Block Attention Module," in *European Conference on Computer Vision (ECCV)*, pp. 3–19, 2018, doi: 10.1007/978-3-030-01234-2_1.

[9] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7132–7141, 2018, doi: 10.1109/CVPR.2018.00745.

[10] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," in *IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626, 2017, doi: 10.1109/ICCV.2017.74.

[11] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 6023–6032, 2019, doi: 10.1109/ICCV.2019.00612.

[12] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond Empirical Risk Minimization," in *International Conference on Learning Representations (ICLR)*, pp. 1–13, 2018.

[13] K. P. Ferentinos, "Deep Learning Models for Plant Disease Detection and Diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018, doi: 10.1016/j.compag.2018.01.009.

[14] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep Learning in Agriculture: A Survey," *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018, doi: 10.1016/j.compag.2018.02.016.

[15] G. Geetharamani and J. Arun Pandian, "Identification of Plant Leaf Diseases Using a Nine-Layer Deep Convolutional Neural Network," *Computers & Electrical Engineering*, vol. 76, pp. 323–338, 2019, doi: 10.1016/j.compeleceng.2019.04.011.